

5

Control Statements: Part 2

OBJECTIVES

- To use the for and do...while repetition statements.
- To use the switch selection statement.
- To use the break and continue program control statements.
- To use the logical operators.

5.3 for Repetition Statement (Cont.)

- **General form of the `for` statement**

- `for` (*initialization*; *loopContinuationCondition*; *increment*)
 statement;

- **General form of the `while` statement**

- *initialization*;
 `while` (*loopContinuationCondition*)
 {
 statement;
 increment;
 }

```

1 // Fig. 5.1: fig05_01.cpp
2 // Counter-controlled repetition.
3 #include <iostream>
4 using std::cout;
5 using std::endl;
6
7 int main()
8 {
9     int counter = 1; // declare and initialize control variable
10
11     while ( counter <= 10 ) // loop-continuation condition
12     {
13         cout << counter << " ";
14         counter++; // increment control variable by 1
15     } // end while
16
17     cout << endl; // output a newline
18     return 0; // successful termination
19 } // end main

```

Outline

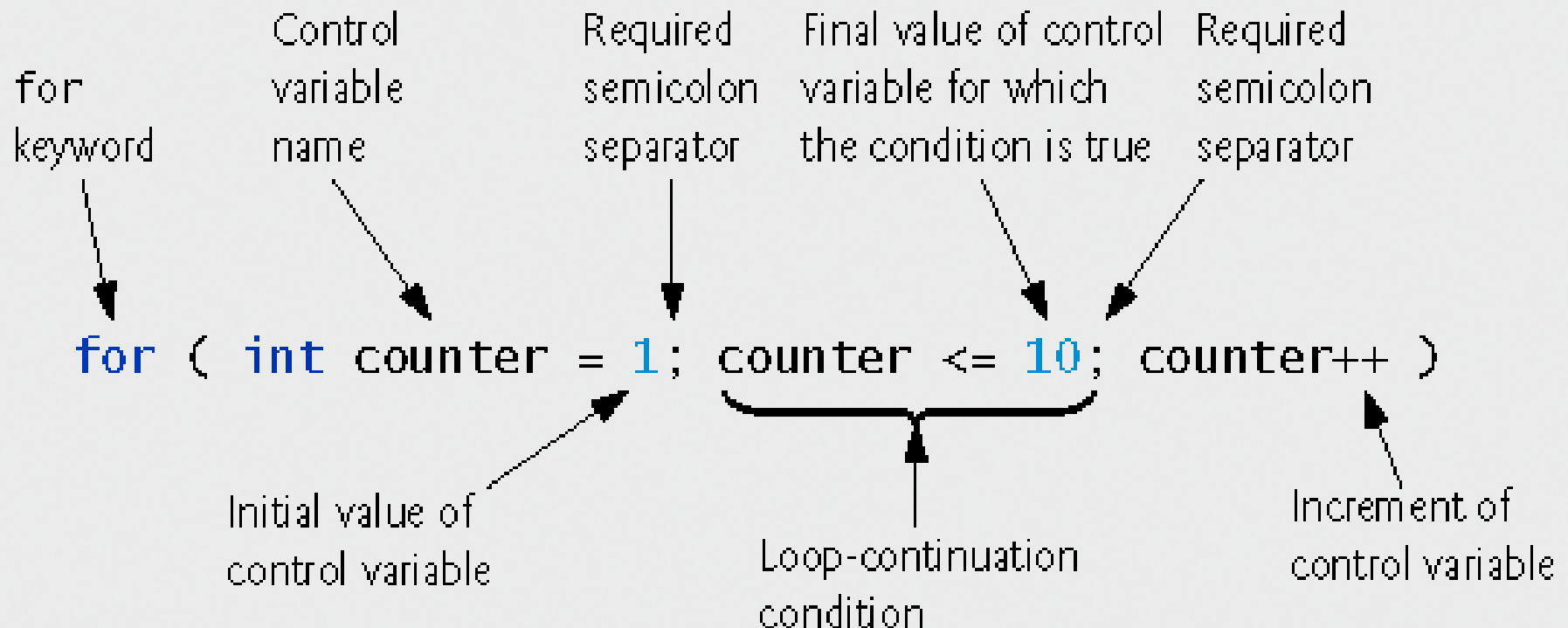
fig05_01.cpp

(1 of 1)

1 2 3 4 5 6 7 8 9 10

5.3 for Repetition Statement

- **for repetition statement**



```
1 // Fig. 5.2: fig05_02.cpp
2 // Counter-controlled repetition with the for statement.
3 #include <iostream>
4 using std::cout;
5 using std::endl;
6
7 int main()
8 {
9     // for statement header includes initialization,
10    // loop-continuation condition and increment.
11    for ( int counter = 1; counter <= 10; counter++ )
12        cout << counter << " ";
13
14    cout << endl; // output a newline
15    return 0; // indicate successful termination
16 } // end main
```

Outline

fig05_02.cpp

(1 of 1)

```
1 2 3 4 5 6 7 8 9 10
```

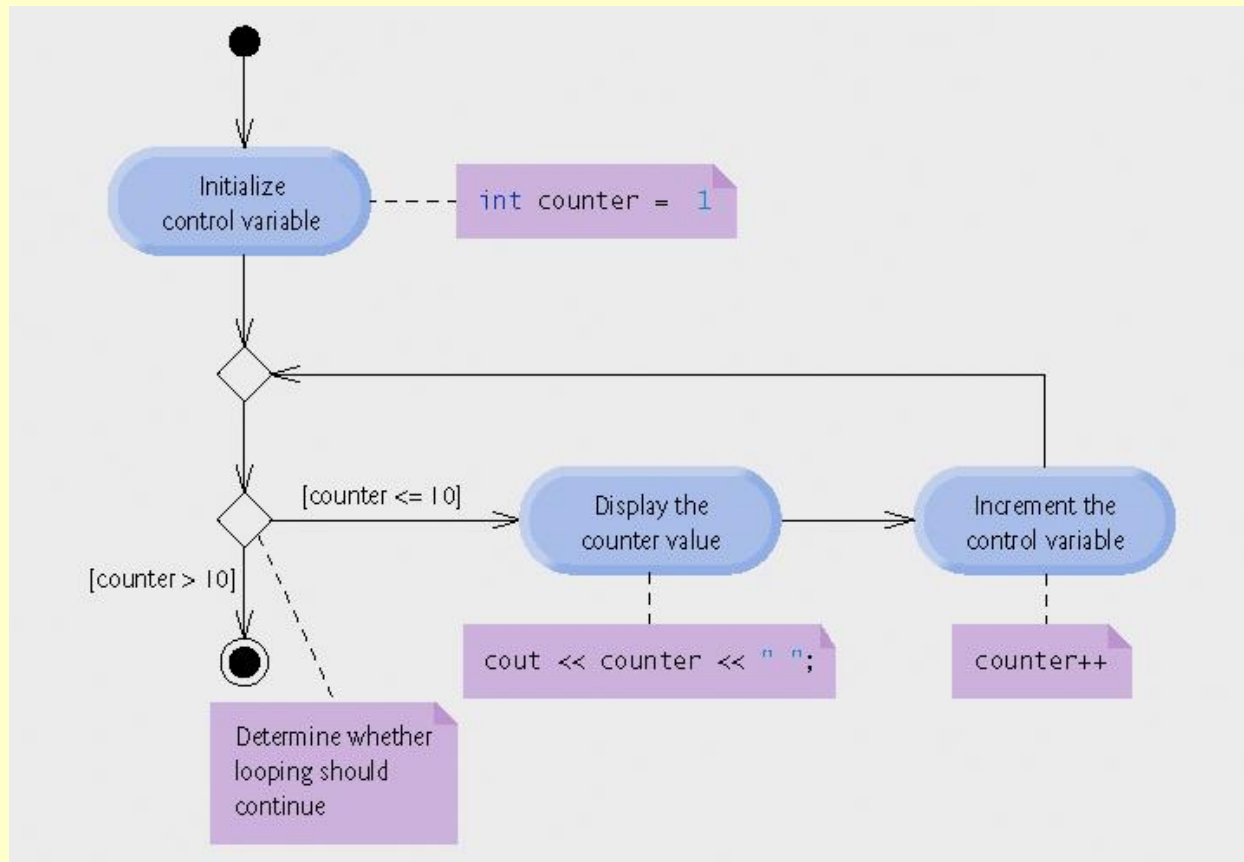
5.3 for Repetition Statement (Cont.)

Line 11: `for ( int counter = 1; counter <= 10; counter++)`

- If the control variable is declared in the *initialization* expression (e.g., `counter`), it will be unknown outside the `for` statement.

5.3 for Repetition Statement (Cont.)

- UML activity diagram



5.4 Examples Using the for Statement

- **for statement examples**

- Vary control variable from 1 to 100 in increments of 1
`for ( int i = 1; i <= 100; i++ )`
- Vary control variable from 100 to 1 in increments of -1
`for ( int i = 100; i >= 1; i-- )`
- Vary control variable from 7 to 77 in steps of 7
`for ( int i = 7; i <= 77; i += 7 )`

Class Work

- Vary control variable from 20 to 2 in steps of -2

Answer:

- Vary control variable over the sequence: 2, 5, 8, 11, 14, 17, 20

Answer:

- Vary control variable over the sequence: 99, 88, 77, 66, 55, 44, 33, 22, 11, 0

Answer:

```
1 // Fig. 5.5: fig05_05.cpp
2 // Summing integers with the for statement.
3 #include <iostream>
4 using std::cout;
5 using std::endl;
6
7 int main()
8 {
9     int total = 0; // initialize total
10
11     // total even integers from 2 through 20
12     for ( int number = 2; number <= 20; number += 2 )
13         total += number;
14
15     cout << "Sum is " << total << endl; // display results
16     return 0; // successful termination
17 } // end main
```

```
Sum is 110
```

Outline

fig05_05.cpp

(1 of 1)

5.4 Examples Using the for Statement (Cont.)

- **Using a comma-separated list of expressions**

- Lines 12-13 of Fig. 5.5 can be rewritten as

```
for ( int number = 2; // initialization
      number <= 20; // loop continuation condition
      total += number, number += 2 ) // total and
                                     // increment
    ; // empty statement
```

```

1 // Fig. 5.6: fig05_06.cpp
2 // Compound interest calculations with for.
3 #include <iostream>
4 using std::cout;
5 using std::endl;
6 using std::fixed;
7
8 #include <iomanip>
9 using std::setw; // enables program to set a field width
10 using std::setprecision;
11
12 #include <cmath> // standard C++ math library
13 using std::pow; // enables program to use function pow
14
15 int main()
16 {
17     double amount; // amount on deposit at end of each year
18     double principal = 1000.0; // initial amount before interest
19     double rate = .05; // interest rate
20
21     // display headers
22     cout << "Year" << setw( 21 ) << "Amount on deposit" << endl;
23
24     // set floating-point number format
25     cout << fixed << setprecision( 2 );
26

```

Outline

fig05_06.cpp

(1 of 2)

```

27 // calculate amount on deposit for each of ten years
28 for ( int year = 1; year <= 10; year++ )
29 {
30     // calculate new amount for specified year
31     amount = principal * pow( 1.0 + rate, year );
32
33     // display the year and the amount
34     cout << setw( 4 ) << year << setw( 21 ) << amount << endl;
35 } // end for
36
37 return 0; // indicate successful termination
38 } // end main

```

Year	Amount on deposit
1	1050.00
2	1102.50
3	1157.63
4	1215.51
5	1276.28
6	1340.10
7	1407.10
8	1477.46
9	1551.33
10	1628.89

Outline

fig05_06.cpp

(2 of 2)

5.4 Examples Using the for Statement (Cont.)

Line 9: `std::setw;`

- **Formatting numeric output**
 - **Stream manipulator `setw`**
 - **Sets field width.**
 - **Right justified by default**
 - **Stream manipulator `left` to left-justify**
 - **Stream manipulator `right` to right-justify**
 - **Applies only to the next output value.**

5.4 Examples Using the for Statement (Cont.)

```
Line 12: include <cmath>  
Line 13: std::pow;
```

- **Standard library function `std::pow`**
 - Calculates an exponent.
 - Example
 - `pow( x, y )`
 - Calculates the value of `x` raised to the `yth` power.
 - Requires header file `<cmath>`.

5.4 Examples Using the for Statement (Cont.)

```
Line 22: cout << "Year" << setw( 21 ) << "Amount on deposit" << endl;
```

- **Specify that the next value output should appear in a field width of 21.**

5.5 do...while Repetition Statement

- **do...while statement**
 - Similar to **while** statement
 - Tests loop-continuation after performing body of loop.
 - Loop body always executes at least once.
- **General form of the while statement**
 - *initialization*;
 do
 {
 statement;
 increment;
 } **while** (*loopContinuationCondition*)

```

1 // Fig. 5.7: fig05_07.cpp
2 // do...while repetition statement.
3 #include <iostream>
4 using std::cout;
5 using std::endl;
6
7 int main()
8 {
9     int counter = 1; // initialize counter
10
11     do
12     {
13         cout << counter << " "; // display counter
14         counter++; // increment counter
15     } while ( counter <= 10 ); // end do...while
16
17     cout << endl; // output a newline
18     return 0; // indicate successful termination
19 } // end main

```

Outline

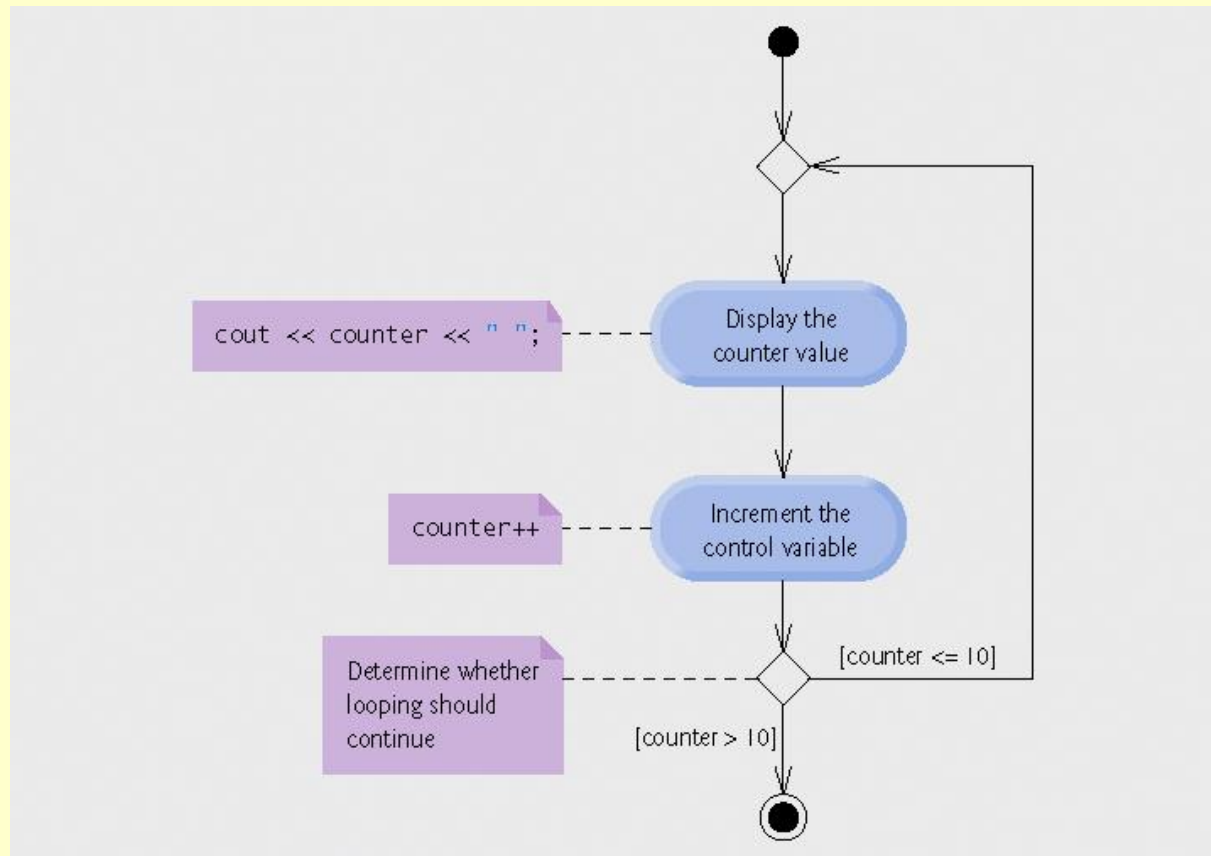
fig05_07.cpp

(1 of 1)

```
1 2 3 4 5 6 7 8 9 10
```

5.5 do...while Repetition Statement

- **UML activity diagram**

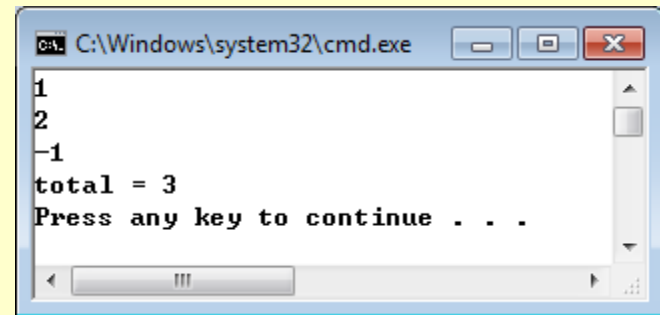


Class Work

- Rewrite the following program by using “do ... while” statement:

```
#include<iostream>
using std::cin;
using std::cout;
```

```
int main() {
    int i = 0, total = 0;
    cin >> i;
    while (i != -1) {
        total += i;
        cin >> i;
    }
    cout << "total = " << total << "\n";
    return 0;
}
```



Class Work (Your work)

Class Work (Answer)

5.6 `switch` Multiple-Selection Statement

- **`switch` statement**

- Used for multiple selections
- Tests a variable or expression.
 - Compare against constant integral expressions to decide on action to take.

5.6 switch Multiple-Selection Statement

- General form of the `switch` statement

- `switch` (*expression*)
{

- `case` *controlling expression*:

- `statement`;

- `break`;

- ...

- `default` *controlling expression*:

- `statement`;

- `break`;

- }

Leave the `switch` Statement.

Without it, execution will go to the next case label.

Executes if no matching case label is found. It is optional.

```

1 // Fig. 5.9: GradeBook.h
2 // Definition of class GradeBook that counts A, B, C, D and F grades.
3 // Member functions are defined in GradeBook.cpp
4
5 #include <string> // program uses C++ standard string class
6 using std::string;
7
8 // GradeBook class definition
9 class GradeBook
10 {
11 public:
12     GradeBook( string ); // constructor initializes course name
13     void setCourseName( string ); // function to set the course name
14     string getCourseName(); // function to retrieve the course name
15     void displayMessage(); // display a welcome message
16     void inputGrades(); // input arbitrary number of grades from user
17     void displayGradeReport(); // display a report based on the grades
18 private:
19     string courseName; // course name for this GradeBook
20     int aCount; // count of A grades
21     int bCount; // count of B grades
22     int cCount; // count of C grades
23     int dCount; // count of D grades
24     int fCount; // count of F grades
25 }; // end class GradeBook

```

Outline

fig05_09.cpp

(1 of 1)

Outline

fig05_10.cpp

(1 of 5)

```
1 // Fig. 5.10: GradeBook.cpp
2 // Member-function definitions for class GradeBook that
3 // uses a switch statement to count A, B, C, D and F grades.
4 #include <iostream>
5 using std::cout;
6 using std::cin;
7 using std::endl;
8
9 #include "GradeBook.h" // include definition of class GradeBook
10
11 // constructor initializes courseName with string supplied as argument;
12 // initializes counter data members to 0
13 GradeBook::GradeBook( string name )
14 {
15     setCourseName( name ); // validate and store courseName
16     aCount = 0; // initialize count of A grades to 0
17     bCount = 0; // initialize count of B grades to 0
18     cCount = 0; // initialize count of C grades to 0
19     dCount = 0; // initialize count of D grades to 0
20     fCount = 0; // initialize count of F grades to 0
21 } // end GradeBook constructor
22
```

```

23 // function to set the course name; limits name to 25 or fewer characters
24 void GradeBook::setCourseName( string name )
25 {
26     if ( name.length() <= 25 ) // if name has 25 or fewer characters
27         courseName = name; // store the course name in the object
28     else // if name is longer than 25 characters
29     { // set courseName to first 25 characters of parameter name
30         courseName = name.substr( 0, 25 ); // select first 25 characters
31         cout << "Name \"\" << name << "\" exceeds maximum length (25).\\n"
32             << "Limiting courseName to first 25 characters.\\n" << endl;
33     } // end if...else
34 } // end function setCourseName
35
36 // function to retrieve the course name
37 string GradeBook::getCourseName()
38 {
39     return courseName;
40 } // end function getCourseName
41
42 // display a welcome message to the GradeBook user
43 void GradeBook::displayMessage()
44 {
45     // this statement calls getCourseName to get the
46     // name of the course this GradeBook represents
47     cout << "Welcome to the grade book for\\n" << getCourseName() << "!\\n"
48         << endl;
49 } // end function displayMessage
50

```

Outline

fig05_10.cpp

(2 of 5)

```

51 // input arbitrary number of grades from user; update grade counter
52 void GradeBook::inputGrades()
53 {
54     int grade; // grade entered by user
55
56     cout << "Enter the letter grades." << endl
57          << "Enter the EOF character to end input." << endl;
58
59     // loop until user types end-of-file key sequence
60     while ( ( grade = cin.get() ) != EOF )
61     {
62         // determine which grade was entered
63         switch ( grade ) // switch statement nested in while
64         {
65             case 'A': // grade was uppercase A
66             case 'a': // or lowercase a
67                 aCount++; // increment aCount
68                 break; // necessary to exit switch
69
70             case 'B': // grade was uppercase B
71             case 'b': // or lowercase b
72                 bCount++; // increment bCount
73                 break; // exit switch
74
75             case 'C': // grade was uppercase C
76             case 'c': // or lowercase c
77                 cCount++; // increment cCount
78                 break; // exit switch
79

```

Outline

fig05_10.cpp

(3 of 5)

```

80     case 'D': // grade was uppercase D
81     case 'd': // or lowercase d
82         dCount++; // increment dCount
83         break; // exit switch
84
85     case 'F': // grade was uppercase F
86     case 'f': // or lowercase f
87         fCount++; // increment fCount
88         break; // exit switch
89
90     case '\n': // ignore newlines,
91     case '\t': // tabs,
92     case ' ': // and spaces in input
93         break; // exit switch
94
95     default: // catch all other characters
96         cout << "Incorrect letter grade entered."
97             << " Enter a new grade." << endl;
98         break; // optional; will exit switch anyway
99     } // end switch
100 } // end while
101 } // end function inputGrades

```

Outline

fig05_10.cpp

(4 of 5)

```

102
103// display a report based on the grades entered by user
104void GradeBook::displayGradeReport()
105{
106    // output summary of results
107    cout << "\n\nNumber of students who received each letter grade:"
108        << "\nA: " << aCount // display number of A grades
109        << "\nB: " << bCount // display number of B grades
110        << "\nC: " << cCount // display number of C grades
111        << "\nD: " << dCount // display number of D grades
112        << "\nF: " << fCount // display number of F grades
113        << endl;
114} // end function displayGradeReport

```

Outline

fig05_01.cpp

(5 of 5)

```

1 // Fig. 5.11: fig05_11.cpp
2 // Create GradeBook object, input grades and display grade report.
3
4 #include "GradeBook.h" // include definition of class GradeBook
5
6 int main()
7 {
8     // create GradeBook object
9     GradeBook myGradeBook( "CS101 C++ Programming" );
10
11     myGradeBook.displayMessage(); // display welcome message
12     myGradeBook.inputGrades(); // read grades from user
13     myGradeBook.displayGradeReport(); // display report based on grades
14     return 0; // indicate successful termination
15 } // end main

```

Outline

fig05_11.cpp

(1 of 2)

Welcome to the grade book for
CS101 C++ Programming!

Enter the letter grades.
Enter the EOF character to end input.

a
B
C
C
A
d
f
C
E
Incorrect letter grade entered. Enter a new grade.
D
A
b
^Z

Number of students who received each letter grade:

A: 3
B: 2
C: 3
D: 2
F: 1

Outline

fig05_11.cpp

(2 of 2)

5.6 switch Multiple-Selection Statement (Cont.)

```
Line 54: int grade;
```

- **Line 54 is used to declare a character variable.**
- **Integer data types**
 - **short**
 - Abbreviation of **short int**
 - Minimum range is -32,768 to 32,767
 - **long**
 - Abbreviation of **long int**
 - Minimum range is -2,147,483,648 to 2,147,483,647

5.6 switch Multiple-Selection Statement (Cont.)

- **int**
 - Equivalent to either **short** or **long** on most computers
- **char**
 - Can be used to represent small integers
- **A computer identifies a character as an integer.**
 - For example, 97 represents the lowercase letter 'a'.
 - Most computers today use ASCII.
 - ASCII = American Standard Code for Information Interchange.

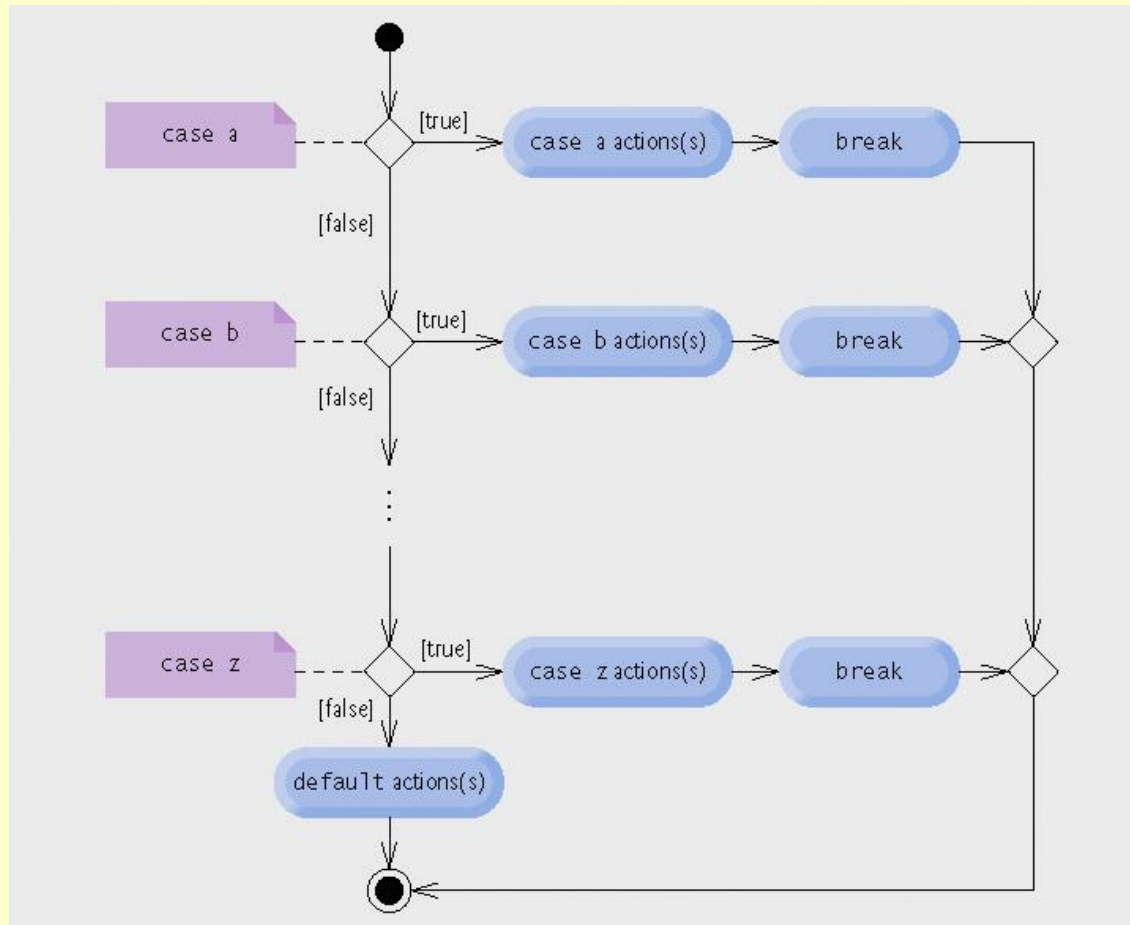
5.6 switch Multiple-Selection Statement (Cont.)

```
Line 60: while ( ( grade = cin.get() ) != EOF )
```

- **Reading character input**
 - Function `cin.get()` reads one character from the keyboard.
 - **EOF = End-Of-File**
 - *<ctrl> d* in **UNIX/Linux**
 - *<ctrl> z* in **Windows**

5.6 switch Multiple-Selection Statement (Cont.)

- **UML activity diagram**



Class Work

- By using switch statement, write a small program to solve the following problem:

Prompt a message “Enter a small letter” and get a character from the user. If the character is ‘a’, ‘e’, ‘i’, ‘o’, or ‘u’, the program displays a message “It is a vowel”. If the input is 0, the message should be “It is not a letter”. If the character is other character, the message should be “It is a consonant”.

Class Work (Your work)

Class Work (Your work)

Class Work (Answer)

5.7 break and continue Statements

- **break/continue statements**

- Alter flow of control.

- **break statement**

- Causes immediate exit from control structure.
- Used in `while`, `for`, `do...while` or `switch` statements

5.7 break and continue Statements

- **continue statement**

- Skips remaining statements in loop body.
 - Proceeds to increment and condition test in **for** loops.
 - Proceeds to condition test in **while/do...while** loops.
- Then performs next iteration (if not terminating).
- Used in **while**, **for** or **do...while** statements

Outline

fig05_13.cpp

(1 of 1)

```
1 // Fig. 5.13: fig05_13.cpp
2 // break statement exiting a for statement.
3 #include <iostream>
4 using std::cout;
5 using std::endl;
6
7 int main()
8 {
9     int count; // control variable also used after loop terminates
10
11     for ( count = 1; count <= 10; count++ ) // loop 10 times
12     {
13         if ( count == 5 )
14             break; // break loop only if x is 5
15
16         cout << count << " ";
17     } // end for
18
19     cout << "\nBroke out of loop at count = " << count << endl;
20     return 0; // indicate successful termination
21 } // end main
```

```
1 2 3 4
Broke out of loop at count = 5
```

```

1 // Fig. 5.14: fig05_14.cpp
2 // continue statement terminating an iteration of a for statement.
3 #include <iostream>
4 using std::cout;
5 using std::endl;
6
7 int main()
8 {
9     for ( int count = 1; count <= 10; count++ ) // loop 10 times
10    {
11        if ( count == 5 ) // if count is 5,
12            continue; // skip remaining code in loop
13
14        cout << count << " ";
15    } // end for
16
17    cout << "\nUsed continue to skip printing 5" << endl;
18    return 0; // indicate successful termination
19 } // end main

```

```

1 2 3 4 6 7 8 9 10
Used continue to skip printing 5

```

Outline

fig05_14.cpp

(1 of 1)

5.8 Logical Operators

- **Logical operators**
 - Combines simple conditions into complex conditions
- **C++ logical operators**
 - **&&** (logical AND)
 - **||** (logical OR)
 - **!** (logical NOT)

5.8 Logical Operators (Cont.)

- **Logical AND (&&) Operator**

- Consider the following `if` statement

```
if ( (gender == 1) && (age >= 65) )  
    seniorFemales++;
```

- Combined condition is `true`
 - If and only if both simple conditions are `true`
 - Combined condition is `false`
 - If either or both of the simple conditions are `false`

5.8 Logical Operators (Cont.)

- **Logical OR (| |) Operator**

- Consider the following **if** statement

```
if ( ( semesterAverage >= 90 )  
    || ( finalExam >= 90 ) )  
    cout << "Student grade is A" << endl;
```

- Combined condition is **true**
 - If either or both of the simple conditions are **true**
- Combined condition is **false**
 - If both of the simple conditions are **false**

5.8 Logical Operators (Cont.)

- **Short-Circuit Evaluation of Complex Conditions**
 - Parts of an expression containing **&&** or **||** operators are evaluated only until it is known whether the condition is true or false.
 - Example
 - `( gender == 1 ) && ( age >= 65 )`
 - Stops immediately if `gender` is not equal to 1.
 - Since the left-side is `false`, the entire expression must be `false`.

5.8 Logical Operators (Cont.)

- **Logical Negation (!) Operator**

- Unary operator
- Returns **true** when its operand is **false**, and vice versa.
- Example

```
if ( !( grade == sentinelValue ) )  
    cout << "The next grade is " << grade << endl;
```

is equivalent to:

```
if ( grade != sentinelValue )  
    cout << "The next grade is " << grade << endl;
```

5.8 Logical Operators (Cont.)

- Stream manipulator `boolalpha`
 - Display `bool` expressions in words, “true” or “false”.

```

1 // Fig. 5.18: fig05_18.cpp
2 // Logical operators.
3 #include <iostream>
4 using std::cout;
5 using std::endl;
6 using std::boolalpha; // causes bool values to print as "true" or "false"
7
8 int main()
9 {
10     // create truth table for && (logical AND) operator
11     cout << boolalpha << "Logical AND (&&)"
12         << "\nfalse && false: " << ( false && false )
13         << "\nfalse && true: " << ( false && true )
14         << "\ntrue && false: " << ( true && false )
15         << "\ntrue && true: " << ( true && true ) << "\n\n";
16
17     // create truth table for || (logical OR) operator
18     cout << "Logical OR (||)"
19         << "\nfalse || false: " << ( false || false )
20         << "\nfalse || true: " << ( false || true )
21         << "\ntrue || false: " << ( true || false )
22         << "\ntrue || true: " << ( true || true ) << "\n\n";
23
24     // create truth table for ! (logical negation) operator
25     cout << "Logical NOT (!)"
26         << "\n!false: " << ( !false )
27         << "\n!true: " << ( !true ) << endl;
28     return 0; // indicate successful termination
29 } // end main

```

Outline

fig05_18.cpp

(1 of 2)

```
Logical AND (&&)
false && false: false
false && true: false
true && false: false
true && true: true
```

```
Logical OR (||)
false || false: false
false || true: true
true || false: true
true || true: true
```

```
Logical NOT (!)
!false: true
!true: false
```

5.9 Confusing Equality (==) and Assignment (=) Operators

- **Accidentally swapping the operators == (equality) and = (assignment)**
 - **Common error**
 - Assignment statements produce a value (the value to be assigned).
 - Expressions that have a value can be used for decision.
 - `Zero = false`, `non-zero = true`
 - **Does not typically cause syntax errors.**
 - Some compilers issue a warning when `=` is used in a context normally expected for `==`.

5.9 Confusing Equality (==) and Assignment (=) Operators (Cont.)

- **Example**

```
if ( payCode == 4 )  
    cout << "You get a bonus!" << endl;
```

- If paycode is 4, bonus is given.

- **If == was replaced with =**

```
if ( payCode = 4 )  
    cout << "You get a bonus!" << endl;
```

- paycode is set to 4 (no matter what it was before).
- Condition is true (since 4 is non-zero).
- Bonus is always given.

5.10 Structured Programming Summary

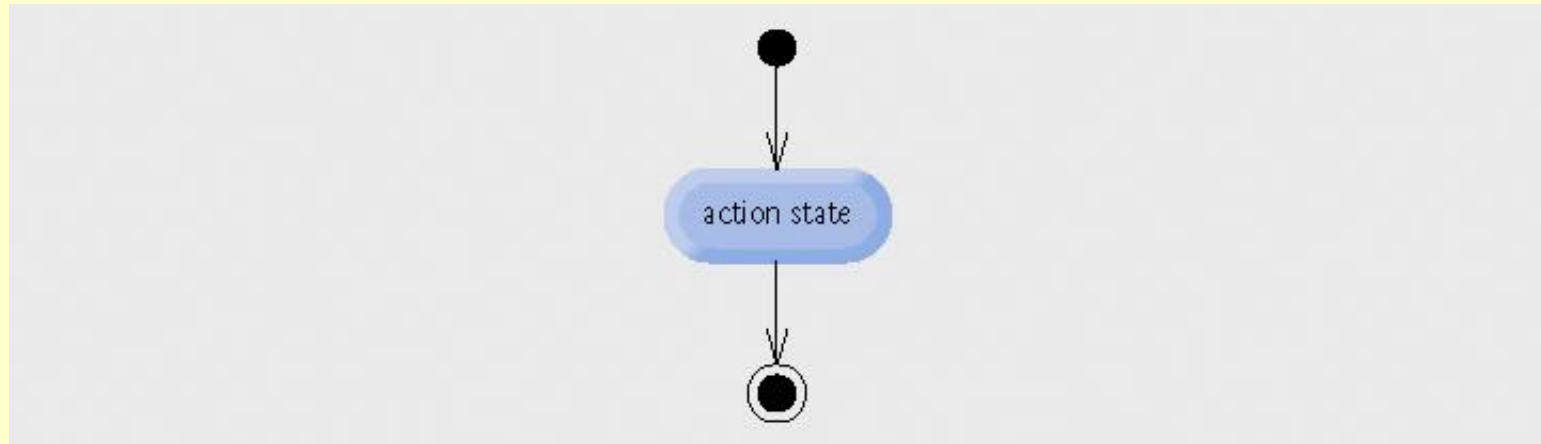
- **Structured programming**
 - Produces programs that are easier to understand, test, debug and modify.

Rules for Forming Structured Programs

- 1) Begin with the “simplest activity diagram” (Fig. 5.22).
- 2) Any action state can be replaced by two action states in sequence.
- 3) Any action state can be replaced by any control statement (sequence, `if`, `if...else`, `switch`, `while`, `do...while` or `for`).
- 4) Rules 2 and 3 can be applied as often as you like and in any order.

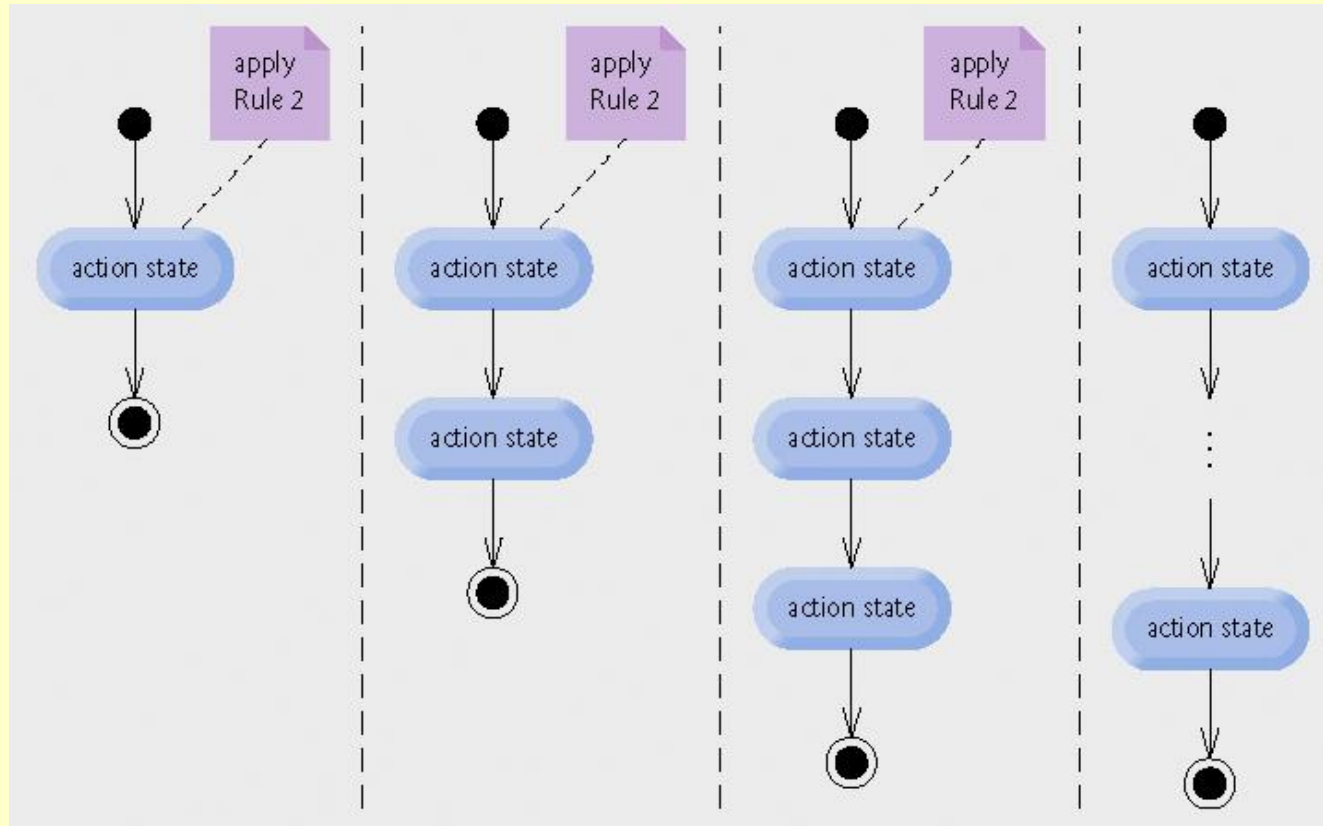
5.10 Structured Programming Summary

- **Rule 1**



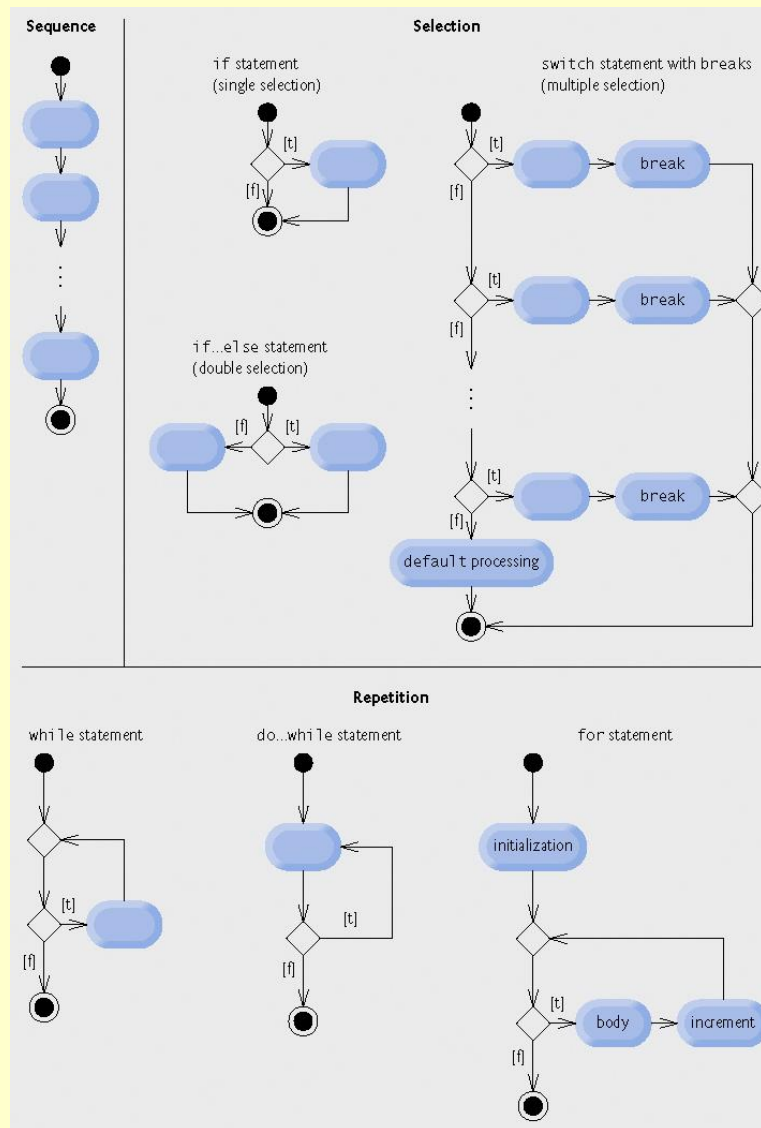
5.10 Structured Programming Summary

- **Rule 2**



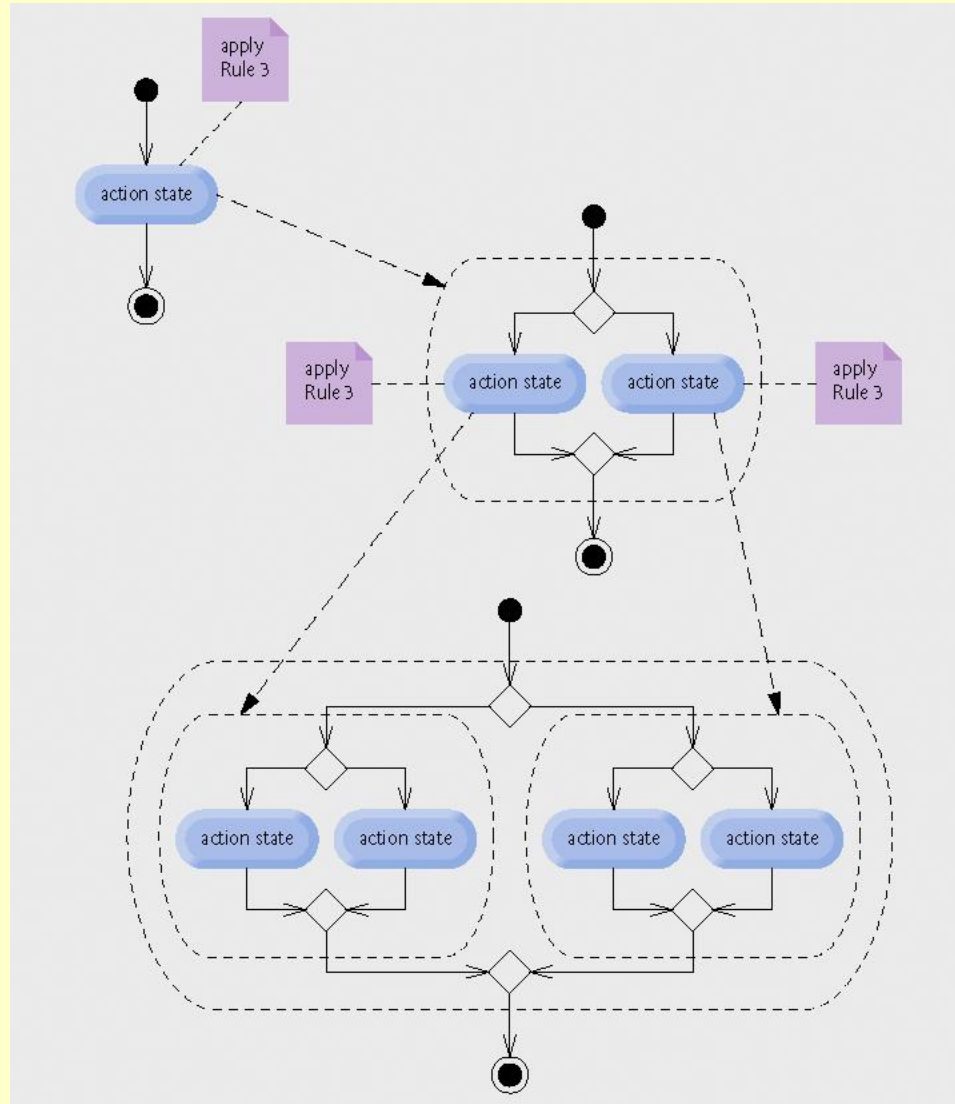
5.10 Structured Programming Summary

- Rule 3



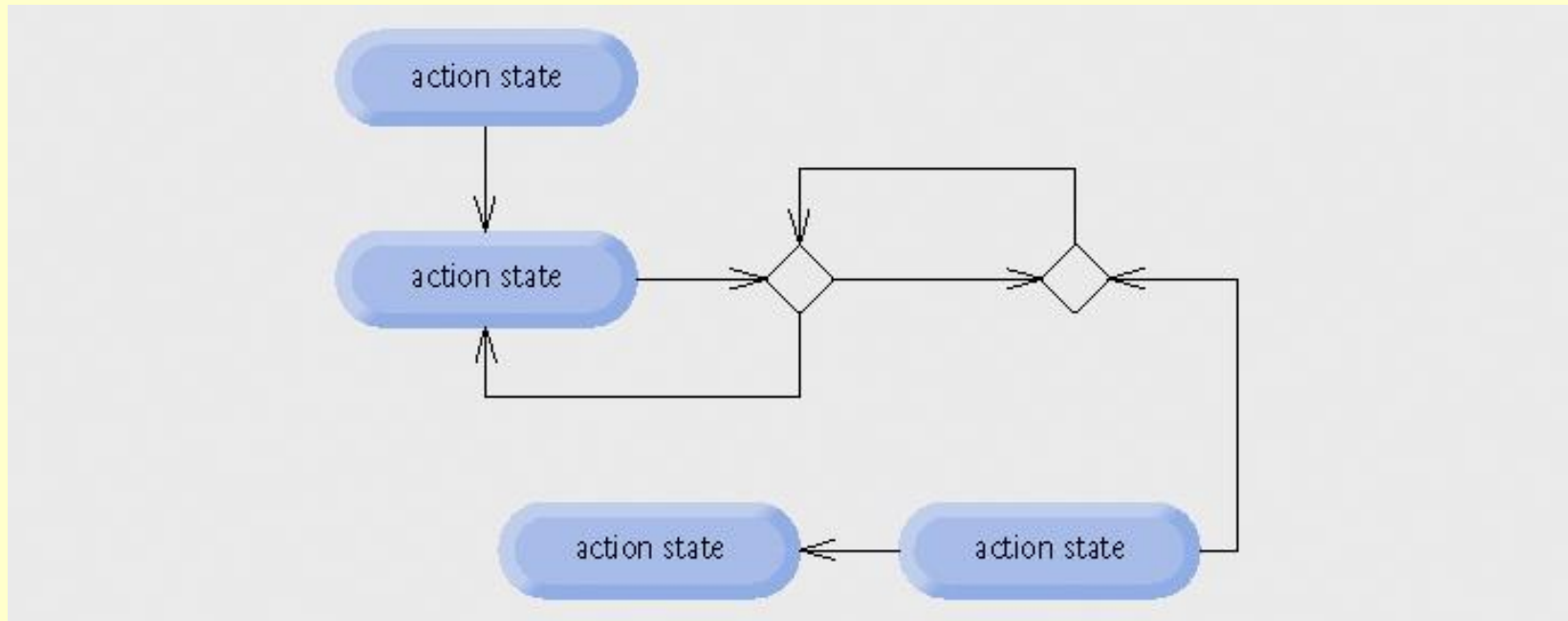
5.10 Structured Programming Summary

- **Rule 4**



5.10 Structured Programming Summary (Cont.)

- **Illegal activity diagram**



Until Next Time

- **Review Chapter 5**
- **Preview Chapter 6**